

spectrogram-self-mark

Mục tiêu bài lab

Bài lab này hướng dẫn sinh viên phân tích một file WAV bằng spectrogram để tìm mẫu tự đánh dấu nằm trong miền tần số theo thời gian.

- Task 1: Kiểm tra hidden.wav là file WAV hợp lệ.
- Task 2: Hoàn thiện spectrogram.py để tạo spectrogram.png.
- Task 3: Quan sát spectrogram.png và ghi thông điệp nhìn thấy vào answer.txt.
- Task 4: Giải thích self-mark nằm trong spectrogram/miền tần số.

Lưu ý: Hướng dẫn này không ghi sẵn đáp án thông điệp ẩn. Sinh viên phải tự tạo spectrogram.png và quan sát ảnh để trả lời.

Các file có sẵn

```
README.md
hidden.wav
spectrogram.py
answer.txt
explanation.txt
check_work.sh
```

File answer.txt và explanation.txt ban đầu để trống. Sinh viên sẽ tự điền sau khi tạo và quan sát spectrogram.

Bước 1: Kiểm tra file audio

Chạy lệnh sau trong container student:

```
file hidden.wav
```

Nếu file hợp lệ, kết quả sẽ cho biết đây là file WAVE audio.

Bước 2: Hoàn thiện spectrogram.py

Mở file spectrogram.py:

```
nano spectrogram.py
```

2.1. Sửa hàm normalize_audio

Thay phần TODO của hàm normalize_audio bằng đoạn code sau:

```
def normalize_audio(data):
    if data.ndim > 1:
        data = data.mean(axis=1)

    if data.dtype == np.int16:
        return data.astype(np.float32) / 32768.0

    if data.dtype == np.int32:
        return data.astype(np.float32) / 2147483648.0
```

```

if data.dtype == np.uint8:
    return (data.astype(np.float32) - 128.0) / 128.0

data = data.astype(np.float32)
max_abs = np.max(np.abs(data))
if max_abs > 0:
    data = data / max_abs

return data

```

2.2. Sửa hàm create_spectrogram

Thay phần TODO của hàm create_spectrogram bằng đoạn code sau:

```

def create_spectrogram(sample_rate, audio, output_png):
    plt.figure(figsize=(12, 4.5), dpi=140)
    plt.specgram(
        audio,
        NFFT=1024,
        Fs=sample_rate,
        noverlap=768,
        cmap="magma",
    )
    plt.ylim(0, 10000)
    plt.xlabel("Time (s)")
    plt.ylabel("Frequency (Hz)")
    plt.title("Hidden spectrogram self-mark")
    plt.colorbar(label="Intensity (dB)")
    plt.tight_layout()
    plt.savefig(output_png)
    plt.close()

```

Bước 3: Tạo ảnh spectrogram

Chạy lệnh:

```
python3 spectrogram.py hidden.wav spectrogram.png
```

Nếu đúng, chương trình sẽ in:

```

SPECTROGRAM_OK
input=hidden.wav
output=spectrogram.png

```

Lúc này file spectrogram.png mới được tạo ra.

Bước 4: Quan sát thông điệp ẩn

Mở spectrogram.png bằng trình xem ảnh. Nếu container không có giao diện, copy ảnh ra máy host để xem. Thông điệp ẩn là chữ xuất hiện trong vùng phổ tần số của ảnh.

Ghi đúng thông điệp nhìn thấy vào answer.txt. Chỉ ghi thông điệp, không ghi thêm giải thích.

```
nano answer.txt
```

Bước 5: Viết giải thích self-marking

Mở explanation.txt:

```
nano explanation.txt
```

Viết ngắn gọn rằng self-mark nằm trong spectrogram, tức là trong miền tần số theo thời gian của file audio. Khi nghe bình thường khó phát hiện vì thông điệp được biểu diễn bằng thành phần tần số, không phải lời nói rõ ràng.

Bước 6: Chạy kiểm tra

Chạy:

```
bash check_work.sh
```

Nếu làm đúng toàn bộ, kết quả cần có:

```
TASK1_AUDIO_FILE_PASS
TASK2_SPECTROGRAM_PASS
TASK3_HIDDEN_MESSAGE_PASS
TASK4_EXPLANATION_PASS
ALL_TASKS_PASS
```

Bước 7: Chấm bằng checkwork

Từ terminal Labtainer ngoài container, chạy:

```
checkwork spectrogram-self-mark
```

Kết quả mong muốn:

Task_1_Check_audio_file	Y
Task_2_Generate_spectrogram	Y
Task_3_Find_hidden_message	Y
Task_4_Explain_self_marking	Y
All_tasks_completed	Y